

ATLAS

Security Architecture Specification

Christopher James Specht | Kintsugi Collective Pty Ltd | March 2026

1 Part 1: Threat Model

Atlas collects and processes data of extraordinary sensitivity. This is not step count data or sleep duration. The system handles:

- Childhood sexual, physical, and emotional abuse histories
- Addiction patterns and substance use logs
- Suicidal ideation and self-harm disclosures
- Core beliefs about self-worth and identity
- Relationship and attachment trauma detail
- ACE scores and developmental history
- Internal dialogue - verbatim, longitudinal
- Dissociation frequency and flashback content
- Medication history and psychiatric treatment
- The complete interior architecture of a human being's most private experience

Atlas will need to have locally operated databases, most countries stipulate health data must be stored locally.

- **Supabase** (self-hosted) Postgres database
- **Llama.cpp** inference
- **ShieldGemma** 2B for payload harmful prompts
- **Mem0** for memory RAG and persistent state memory
- **Kong Gateway** for routing and overall backend structure and security layer

Why this data is uniquely dangerous:

Actor	Motivation	Harm potential
Employer	Hiring decisions, performance management, constructive dismissal	Discrimination, job loss, career destruction
Insurer	Life insurance, income protection, health cover pricing	Policy denial, premium loading, exclusion clauses
Family court	Custody proceedings, parenting capacity assessments	Loss of access to children
Abusive ex-partner	Map of vulnerabilities, triggers, fears	Targeted psychological harm, stalking, coercion
Government / law enforcement	Depending on jurisdiction, legal compulsion to produce records	Surveillance, criminalisation of mental health history

Actor	Motivation	Harm potential
Data broker	Commercial sale of health profiles	Permanent loss of control over most private information
Malicious actor / hacker	Blackmail, identity theft, targeted manipulation	Extortion, reputational destruction

1.1 Primary Threat Vectors

Threat	Description	Likelihood	Impact
Data breach: Supabase	Attacker gains access to database contents	Medium: cloud databases are targeted regularly	Critical if data is readable
Data deletion: malicious	Attacker deletes ServerKey or encrypted blobs	Low: requires authenticated access	High: permanent data loss
Supabase access vulnerabilities	Misconfigured RLS, SQL injection, privilege escalation	Medium: common in poorly configured deployments	Critical
Device theft/loss	Physical access to unlocked device	Medium: phones are lost and stolen	High if passcode not required
Legal compulsion	Court order requiring production of user records	Medium: mental health records are legally sought	Severe if data is readable
Insider threat	Rogue Kintsugi Collective employee or contractor	Low but non:zero	Critical if architecture allows it
Lost passcode	User loses 5:digit code, cannot access data	High: users lose passwords regularly	High: permanent lock:out without recovery
Device change	User gets new phone, DeviceKey changes	High: phones are replaced regularly	High without recovery mechanism

2 Part 2: The Sovereign Signal Vault Architecture

Core principle:

Standard security models say: trust us with your data and we will protect it.

Kintsugi Collective says: **we built it so you do not have to trust us.**

- The architecture is designed so that Kintsugi Collective is architecturally incapable of reading user data in plaintext - not just contractually prohibited from doing so.
- This distinction is critical.
- A promise not to read data can be broken under legal pressure, by a rogue employee, or by a change in company policy. An architectural incapability cannot.

2.1 Layer 1 - The Split Key System

All user data is protected by a MasterKey that is never stored anywhere in its complete form. It is derived from two halves that are stored in entirely separate locations - one on the user's device, one in Supabase. Neither half alone can decrypt anything.

```
DeviceKey = PBKDF2(passcode_5digit + device_hardware_id, salt,
310000_iterations, SHA-256)
ServerKey = CSPRNG(256 bits) // generated client-side at registration,
stored in Supabase
MasterKey = HKDF(DeviceKey || ServerKey, salt, SHA-256)
```

Key	Where stored	Who can access	Alone - can it decrypt?
DeviceKey	Device only: EncryptedSharedPreferences (Android Jetpack Security / iOS Secure Enclave)	User only: derived from their passcode + hardware	No: useless without ServerKey
ServerKey	Supabase: against UUID only	Supabase infrastructure: but isolated, no PII link	No: useless without DeviceKey
MasterKey	Nowhere: exists only in device RAM during active session	Nobody: generated on: device each session, never persisted	Yes: but never stored

2.1.1 What a Supabase breach reveals:

An attacker who gains full Supabase access obtains: ServerKey (one half of the key), encrypted blobs (unreadable without MasterKey), UUID (no PII attached), encrypted memories (unreadable).

They cannot derive MasterKey without DeviceKey. They cannot obtain DeviceKey without physical access to the user's device AND the user's 5-digit passcode. The breach yields nothing meaningful.

2.2 Layer 2 - Encryption at the Edge

All health data is encrypted on the device before it is transmitted to Supabase. Supabase never receives plaintext. The database stores only ciphertext blobs it cannot read.

```
Encrypted_payload = AES-256-GCM(MasterKey, plaintext_data, random_nonce)
// Supabase stores: { uuid, encrypted_blob, nonce, timestamp }
// Supabase never stores: diary entries, tensor scores, journal content,
//                               memories, disclosures, or any readable health
data
```

This applies to every data type: diary cards, journal entries, chat history, tensor scores, reflect responses, memories, biological baseline data. Everything that leaves the device is ciphertext.

2.2.1 Consequences for legal pressure:

If Kintsugi Collective receives a court order to produce user records, we can produce: the encrypted blobs (unreadable), the UUID (no PII), the ServerKey (useless without DeviceKey).

We cannot produce readable user data. This is not a refusal - it is a genuine incapability that is architecturally demonstrable. The legal position is: we do not have the data in readable form. We never did.

2.3 Layer 3 – Mem0 Memory Vault

Mem0 memories are the most sensitive data layer - longitudinal, intimate, and irreplaceable. They receive an additional encryption envelope beyond the standard layer.

```
MemoryKey    = HKDF(MasterKey || memory_timestamp || uuid, SHA-256)
Enc_memory   = AES-256-GCM(MemoryKey, memory_content)
```

Each memory chunk has its own derived key. Compromising one memory's key reveals only that memory - it does not compromise any other memory or any other data type.

Before any LLM API call, the app decrypts relevant memories locally on-device, assembles the context in memory, and transmits the composed prompt. The API never receives a reference to the memory store - it receives a session-scoped context that exists only for that exchange.

2.4 Layer 4 - The Prompt Shield

Even with RLS and other embedded measures, prompt content exists transiently in the provider's infrastructure. For the most sensitive disclosures, an additional pseudonymisation layer is applied before transmission.

```
// Sensitive fields are tokenised before leaving the device
Original: 'My stepfather abused me from age 6'
Tokenised: 'Entity_A [caregiver_type_2] caused harm_type_1 to Subject
during developmental_period_1'

// Mapping table lives only on device, encrypted under MasterKey
// API receives semantic structure without literal disclosure content
```

The AI receives sufficient semantic context to respond meaningfully. The literal content of the most sensitive disclosures never leaves the device in plaintext. The tokenisation mapping is personal to each user and stored only in their device-encrypted local store.

2.5 Layer 5 - The Burn Protocol

True deletion is technically difficult - database write-ahead logs, backups, and infrastructure snapshots can retain deleted records. The Sovereign Signal Vault solves this through key destruction rather than data deletion.

```
// Burn sequence:
1. User confirms burn with 5-digit passcode
2. 24-hour delay initiated - email confirmation sent to address on file
3. If confirmed after 24 hours:
   a. ServerKey deleted from Supabase - permanent, cascades to all tables
   b. DeviceKey overwritten with CSPRNG random bytes
   c. MasterKey evaporated from session memory
   d. Encrypted blobs remain in Supabase permanently

// Result: encrypted blobs are now computationally indecipherable
// AES-256-GCM with destroyed key = unrecoverable within universe lifetime
// Data is not deleted - data is made permanently meaningless
```

Why key destruction is stronger than deletion:

Deleted data can sometimes be recovered from database internals, backups, or forensic analysis. Data encrypted with a permanently destroyed key cannot be recovered by any known computational method within any feasible timeframe. Key destruction is cryptographically irreversible in a way that data deletion is not.

The 24-hour delay:

Protects against an attacker who has gained temporary access to a logged-in session attempting to burn the user's data maliciously. The user receives an email and has 24 hours to cancel. The burn can only be completed with passcode confirmation - the email link alone is not sufficient.

2.6 Layer 6 - The Transparency Ledger

Atlas publishes a real-time public ledger of every legal request, government enquiry, or third-party demand for user data received by Kintsugi Collective. This is not a privacy policy claim - it is a public, auditable, timestamped record.

Ledger entry field	Content
Date received	Timestamp of request
Requesting entity type	Court order / law enforcement / civil subpoena / other (no identifying details of requester)
Jurisdiction	Australian federal / state / international
What was requested	Description of data sought
What was provided	Encrypted blobs / UUID only / nothing
Why readable data could not be provided	Architectural incapability statement
Outcome	Request satisfied / partially satisfied / declined / pending

The ledger is published at a public URL. Any user can check whether any requests have been made. This transforms a privacy policy promise into a verifiable public commitment.

2.7 Layer 7 – prompt payload fingerprinting.

This encodes each prompt sent to the back-end services as being authentic. This will require that the UUID is stamped onto the payload prior to leaving the device.

3 Part 3: Redundancies and Failure Mode Responses

Every security architecture has failure modes. The following documents each identified failure scenario and the redundancy mechanism designed to address it.

3.1 Failure Mode 1 - Lost 6-Digit Passcode

Scenario:

User forgets their 6-digit passcode. DeviceKey cannot be derived. MasterKey cannot be generated. Data is inaccessible.

3.1.1 Redundancy - Emergency Recovery Phrase

At profile creation, the system generates a 12-word BIP39-standard recovery phrase. This phrase is shown once, never stored by Kintsugi Collective, and the user is instructed to write it down physically and store it safely.

```
RecoveryKey      = HKDF(bip39_12_words + uuid, SHA-256)
RecoveryMaster   = HKDF(RecoveryKey || ServerKey, SHA-256)

// RecoveryMaster can decrypt data identically to MasterKey
// Recovery phrase is hardware-independent - works on any device
// Kintsugi Collective never stores or transmits the recovery phrase
```

Scenario	Outcome	Recovery available?
Lost passcode, have recovery phrase	Use recovery phrase to regenerate MasterKey. Set new passcode.	Yes - full recovery
Lost passcode, lost recovery phrase	Data permanently inaccessible	No - by design
Lost phone, know passcode	Install app on new device. Enter passcode + recovery phrase to restore.	Yes - full recovery
Lost phone, lost passcode, lost recovery phrase	Data permanently inaccessible	No - by design

User communication at setup:

During the Cyber Privacy onboarding page, the user is shown the recovery phrase and told explicitly:

'This phrase is the only way to recover your data if you lose your passcode or change devices. Kintsugi Collective does not have it. If you lose it and lose your passcode, your data cannot be recovered by anyone. Write it down. Store it somewhere safe that is not your phone.'

The user must manually type back 3 random words from the phrase to confirm they have recorded it before proceeding. No checkbox. Active confirmation only

3.2 Failure Mode 2 - Device Change

Scenario:

User gets a new phone. DeviceKey is derived from device hardware ID - it changes on new hardware. Existing DeviceKey is gone.

3.2.1 Redundancy - Hardware-Independent Recovery

The recovery phrase is hardware-independent by design. It derives RecoveryKey via HKDF without a hardware component. On a new device:

1. Install Atlas on new device
2. Enter UUID (displayed in profile settings on old device before transfer, or recoverable from email used at registration)
3. Enter 12-word recovery phrase
4. System fetches ServerKey from Supabase using UUID
5. RecoveryMaster generated - all data decrypted and accessible
6. User sets new passcode - new DeviceKey generated and bound to new hardware
7. Old DeviceKey is invalidated

3.3 Failure Mode 3 - Malicious ServerKey Deletion

Scenario:

Attacker with Supabase access (via breach or insider threat) deletes the ServerKey. Even with DeviceKey and recovery phrase, data becomes inaccessible because the ServerKey half is gone.

3.3.1 Redundancy - Protected ServerKey with Deletion Delay

ServerKey deletion requires two conditions:

- Authenticated request using UUID + account password (not passcode - separate credential)
- 24-hour delay with email confirmation to address on file

A malicious actor with Supabase access cannot instantly delete a ServerKey. The user receives an email alert within minutes of a deletion request being initiated. If the user did not initiate it, they cancel it via the email link within the 24-hour window.

```
// ServerKey deletion request:  
1. Request logged with timestamp
```

2. Email sent to user immediately: 'A request to delete your account data has been received.'
3. Email contains: CONFIRM link + CANCEL link
4. CANCEL link invalidates the request immediately
5. If not cancelled within 24 hours AND CONFIRM clicked: deletion proceeds
6. Deletion cannot proceed with CONFIRM link alone - requires passcode re-entry in app

Additional protection:

Rate limiting on ServerKey access: maximum 5 failed UUID+password attempts before a 24-hour lockout. Any lockout triggers an email alert to the user. This makes brute-force access to the deletion pathway computationally expensive.

3.4 Failure Mode 4 - Physical Device Compromise

Scenario:

Device is stolen or accessed while the app is unlocked or session is active. Attacker has access to decrypted data in memory.

3.4.1 Redundancy - Session Controls

- Auto-lock: session terminates and MasterKey evaporated from memory after 5 minutes of inactivity
- Biometric re-authentication required to resume a locked session
- Background app: MasterKey evaporated immediately when app enters background
- Failed biometric: falls back to 5-digit passcode. 5 failed passcode attempts triggers 24-hour lockout and email alert.
- Remote session invalidation: user can invalidate all active sessions from any device via account password - new passcode entry required on all devices

4 Part 4: Security Onboarding Flow

The security architecture is introduced to the user progressively during onboarding. Each stage is explained in plain language without technical jargon. The user understands what they are creating and why before they create it.

Stage	Screen	What happens	Security event
1	App Store / Play Store	User downloads app. No account exists yet. No data transmitted.	No security event - app is inert until access code entered
2	Splash - Welcome	Access code entry. Code validated against Supabase invite table. No PII collected yet.	Access code verified. UUID pre-generated and held pending profile creation.
3	Profile Creation	Username, age (not DOB), gender, postcode, email for 2FA and alerts, data policy acceptance Y/N, commitment question Y/N/unsure.	Email stored in Supabase (not linked to health data - separate table). Profile data stored on device only.

Stage	Screen	What happens	Security event
4	UUID Assignment	UUID confirmed and stored in EncryptedSharedPreferences. ServerKey generated client-side and stored in Supabase against UUID only.	Split key architecture initialised. UUID is now the system identity.
5	Recovery Phrase	12-word BIP39 phrase displayed. User types back 3 random words to confirm recording. Phrase never transmitted to Kintsugi Collective.	Recovery phrase established. User holds the only copy.
6	Cyber Privacy Page	5-digit passcode creation. DeviceKey derived. MasterKey generated for first time. Explanation of what the passcode does and cannot be recovered.	Full key architecture active. All subsequent data encrypted under MasterKey.
7	Data Policy Commitment	Plain language explanation: what is collected, what is not, where it lives, what Kintsugi Collective can and cannot access, how to burn all data.	User informed consent documented.
8	Success	Welcome. System ready. Companion introduces itself.	Onboarding complete. Session active.

5 Part 5: Data Architecture - What Lives Where

Data type	Location	Form	Accessible to Kintsugi Collective?	Accessible under legal compulsion?
UUID	Supabase + Device	Plaintext UUID only	Yes - UUID only, no PII attached	Yes - UUID only, meaningless without data link
Username, preferred name	Device only	Encrypted under DeviceKey	No	No - never transmitted
Email address	Supabase - separate isolated table	Plaintext - used only for alerts and 2FA	Yes - for operational necessity only	Yes - email address only
Age, gender, postcode	Device only	Encrypted under DeviceKey	No	No - never transmitted
ServerKey	Supabase - against UUID	Plaintext 256-bit key	Yes - but useless without DeviceKey	Yes - but useless without DeviceKey
DeviceKey	Device only - Secure Enclave	Derived from passcode+hardware, never stored	No	No - never transmitted
MasterKey	Device RAM only during session	Derived in memory, never persisted	No	No - evaporates on session end
Recovery phrase	User holds physically	Never stored digitally by Kintsugi Collective	No	No - never transmitted
All health data (diary, journal, tensor, reflect)	Supabase	AES-256-GCM ciphertext blobs	No - encrypted, unreadable	No - cannot decrypt without MasterKey
ReMe memories	Supabase	AES-256-GCM with per-memory derived key	No - encrypted, unreadable	No - cannot decrypt
Chat history	Supabase	AES-256-GCM ciphertext	No	No
Audio recordings	Supabase Storage - UUID bucket	AES-256-GCM encrypted	No	No
LLM prompt content	Supabase or destroyed			

6 Part 6: Legal Position Statement

Kintsugi Collective Pty Ltd - Formal Position on User Data:

What we can provide under legal compulsion:

- UUID associated with a given email address (if the email address is known to the requesting party)
- Encrypted data blobs associated with that UUID
- ServerKey associated with that UUID
- Metadata: account creation timestamp, last access timestamp

What we cannot provide under any circumstances:

- Readable health data - we do not possess the decryption capability
- DeviceKey - we have never possessed it
- MasterKey - it exists only in device RAM during active sessions and is never transmitted or stored
- Recovery phrase - we have never possessed it

Why we cannot provide readable data:

The Sovereign Signal Vault architecture was deliberately designed so that Kintsugi Collective possesses only one half of the key required to decrypt user data. The other half exists only on the user's device. This is not a procedural policy choice - it is an architectural constraint that cannot be overridden by a court order, a change in company policy, or any other mechanism short of physical access to the user's device and knowledge of their passcode.

We can demonstrate this architecture to any court or regulatory body. The technical implementation is auditable and the architectural constraint is verifiable.

7 Part 7: Apple App Store Review - Security Documentation

Apple's guidelines for apps handling sensitive health and mental health data require specific documentation and architectural evidence. The following addresses each relevant requirement.

Apple requirement	Atlas response
Data collection disclosure	Full disclosure in App Store listing, privacy nutrition label, and in-app data policy page. Explicit enumeration of every data type collected.
Health data consent	Explicit Y/N consent for health data collection during onboarding. Cannot proceed without explicit consent. Documented in onboarding audit log.
Data deletion mechanism	Burn protocol fully implemented. User can delete all data at any time via Profile page. 24-hour delay with email confirmation. Documented and testable.
Crisis resources accessible without authentication	'I need help now' option on login/splash screen - bypasses all authentication and routes directly to crisis resources. Available even to unauthenticated users.
No diagnostic or medical claims	App explicitly states it is not a diagnostic tool in all relevant contexts. Companion system prompt includes this as a hard constraint. Marketing materials reviewed for compliance.
Data retention policy	Documented: health data retained until user deletes it or burns their profile. Email retained for operational necessity only. No data sold or shared with third parties.
Third party SDK disclosure	All third party services disclosed: Supabase (data storage), Groq (STT/TTS - ZDR).

The standard this sets:

Every other mental health app says: trust us with your data.

Atlas says: we built it so you don't have to.

The data collected by Atlas is the most intimate, the most sensitive, and the most dangerous in the wrong hands of any consumer application in existence. The people this app serves have already been failed repeatedly by systems that claimed to hold their information safely. They deserve an architecture that does not require trust - one that makes privacy mathematically verifiable rather than contractually promised.

8 Prompt Shield

8.1 Schema Philosophy

"Preserve signal. Obfuscate identifier. Honour agency."
This schema implements tiered **pseudonymisation**: literal identifiers are tokenised, while therapeutic signal (modality, intensity, somatic valence, relational context) is preserved in structured form for LLM interpretation.

8.2 Token Grammar & Format

```
typescript
[Token_Type]_[Category]_[Tier]_[Optional_Modifier]

Example: [ENT]_[CAREGIVER]_[T1]_[MALE]
         [HARM]_[EMOTIONAL]_[T2]_[CHRONIC]
         [DEV]_[AGE_RANGE]_[T1]_[EARLY_CHILD]
[Token_Type]_[Category]_[Tier]_[Optional_Modifier]
Example: [ENT]_[CAREGIVER]_[T1]_[MALE]
         [HARM]_[EMOTIONAL]_[T2]_[CHRONIC]
         [DEV]_[AGE_RANGE]_[T1]_[EARLY_CHILD]
```

Token Components

Component	Format	Description
TOKEN_TYPE	3-letter code	Entity, Harm, Developmental, Temporal, Somatic, Relational, Cognitive
CATEGORY	snake_case	Specific semantic class (e.g., caregiver, abuse_type, body_region)
TIER	T1/T2/T3	Privacy tier (see §3)
MODIFIER	optional	Contextual nuance (e.g., chronic, acute, recurring)

8.3 Tiered Tokenisation Framework

8.3.1 Tier 1 (T1) — High Sensitivity: Full Pseudonymisation

Identifiers that could directly re-identify or cause harm if exposed.

Category	Token Pattern	Example Original > Tokenised
Persons	[ENT]_[ROLE]_[T1]_[OPT_DEMO]	"my stepfather" > [ENT]_[CAREGIVER]_[T1]_[MALE]
Locations	[LOC]_[TYPE]_[T1]	"the basement at 12 Oak St" > [LOC]_[RESIDENTIAL]_[T1]
Dates/Ages	[TIME]_[PRECISION]_[T1]	"age 6, March 2003" > [TIME]_[AGE_RANGE]_[T1]_[EARLY_CHILD]
Institutions	[ORG]_[TYPE]_[T1]	"St. Mary's Hospital" > [ORG]_[MEDICAL]_[T1]
Unique Events	[EVT]_[UNIQUE]_[T1]	"the night of the fire" > [EVT]_[TRAUMA_INDEX]_[T1]_[IDX_001]

8.3.2 Tier 2 (T2) — Medium Sensitivity: Semantic Preservation

Harm modalities, relational dynamics, somatic patterns — preserved for therapeutic response calibration.

Category	Token Pattern	Example Original > Tokenised
Harm Type	[HARM]_[MODALITY]_[T2]_[SEVERITY]	"he would hit me" > [HARM]_[PHYSICAL]_[T2]_[MODERATE]
Emotional Valence	[AFF]_[VALENCE]_[T2]_[INTENSITY]	"I felt utterly worthless" > [AFF]_[SHAME]_[T2]_[HIGH]
Relational Pattern	[REL]_[DYNAMIC]_[T2]	"I always apologize first" > [REL]_[OVER_FUNCTIONING]_[T2]
Somatic Signature	[SOM]_[REGION]_[T2]_[QUALITY]	"my chest gets tight" > [SOM]_[THORAX]_[T2]_[CONSTRICTION]
Cognitive Pattern	[COG]_[DISTORTION]_[T2]	"it's all my fault" > [COG]_[PERSONALISATION]_[T2]

8.4 Security & Resilience Considerations

8.4.1 On-Device Requirements

- Mapping table stored in platform secure enclave (iOS Keychain / Android Keystore)
- All tokenisation occurs pre-network; plaintext literals never enter application memory post-transformation
- MasterKey derived from user passphrase via Argon2id (≥88 bits entropy target)

8.4.2 Loss Mitigation

- Optional user-controlled encrypted backup: mapping table > encrypted blob > user-held storage (e.g., personal cloud, encrypted USB)
- Recovery flow: user re-enters passphrase > derives MasterKey > decrypts backup > restores mapping
- No server-side key escrow. Ever.

8.4.3 Attack Surface Reduction

- Token patterns are user-specific; cross-user correlation impossible without mapping table
- [T1] tokens are opaque identifiers; semantic meaning exists only in local mapping
- API receives structured semantic context, not reversible pseudonyms

8.5 LLM System Prompt Snippet (Token Interpretation Guide)

markdown

Tokenised Input Protocol

When you receive input containing `[TOKEN]` patterns:

1. **Never request clarification** on `[T1]` tokens (entities, locations, unique events).

These are intentionally opaque for user sovereignty.

2. **Respond to `[T2]` semantic clusters** with trauma-informed validation:

- `[HARM]\_[PHYSICAL]\_[T2]\_\*` > acknowledge bodily safety, offer somatic grounding *only if requested*

- `[AFF]\_[SHAME]\_[T2]\_[HIGH]` > reflect without minimising; avoid "it's not your fault" unless user invites reframing

- `[SOM]\_[THORAX]\_[T2]\_[CONSTRICTION]` > note somatic pattern; do not prescribe breathwork unless asked

3. **Use intensity/state cues** (Tier 3) to calibrate response depth:

- `intensity: ≥0.8` > prioritise containment over exploration
- `state: dysregulated` > match tone, reduce cognitive load, avoid multi-part questions

4. **Honour request cues**:

- `request: witness` > reflect, validate, stay. No suggestions.
- `request: pattern` > offer tensor-aware observation ("I notice REL and VIC axes showing drift...")
- `request: resource` > provide widget-style options, user-selectable

5. **Remember**: You are receiving semantic structure, not literal disclosure.

Your role is to respond to the *shape* of the experience, not the *content* of the identifier.

9 Prompt Shield Tokeniser API v0.2

Atlas Layer 4 — Sovereign Disclosure Protection & Tensor-Ready Payload Generation

9.1 Module Architecture Overview

The `PromptShieldTokeniser` operates as a fully on-device, zero-transmission gateway between user input and the LLM intelligence layer. It performs:

1. **NER & Tier Assignment** > Identifies sensitive spans, maps to schema tokens
2. **Semantic Extraction** > Preserves therapeutic signal (modality, intensity, regulatory state)
3. **Tensor Impact Computation** > Estimates axis drift, $\Psi(t)$ load, Ω contribution
4. **Red Zone Evaluation** > Computes $\Phi(t)$ threshold, returns alert state without content
5. **LLM Context Packaging** > Outputs a strictly structured, sovereignty-aligned payload

All literal disclosures remain in volatile secure memory and are zeroed post-processing. Only the tokenised string, semantic context, and numerical tensor metadata traverse the API boundary.

9.2 Core Type Definitions

typescript

```
// Atlas Seven-Axis Enum
type Axis = 'REL' | 'OCCF' | 'EREG' | 'CC' | 'SOM' | 'VIC' | 'NI';

// Privacy Tier Configuration
type PrivacyTier = 'balanced' | 'maximum' | 'full_context';

// Regulatory State (Polyvagal/ND-adapted)
type RegulatoryState = 'regulated' | 'hyperaroused' | 'shutdown' |
  'dissociated' | 'alexithymic_flat';

// Companion Interaction Mode (Ten Commitments aligned)
type CompanionMode = 'witness' | 'pattern' | 'resource' | 'presence' |
  'crisis_consult';

// Token Schema Structure
interface TokenDefinition {
  type: 'ENT' | 'LOC' | 'TIME' | 'HARM' | 'AFF' | 'REL' | 'SOM' | 'COG' |
  'EVT' | 'ORG';
  category: string;
  tier: 'T1' | 'T2' | 'T3';
```

```

    modifier?: string;
}

// Encrypted Mapping Table Entry
interface MappingEntry {
    tokenKey: string; // e.g., "[ENT]_[CAREGIVER]_[T1]_[MALE]"
    literalPool: string[];
    contextHint?: string;
    axisImpact: Partial<Record<Axis, number>>; // -1.0 to 1.0
    omegaWeight: number; // 0.0-1.0 contribution to  $\Omega$ 
    ndProxyFlag?: boolean; // true if mapped via external/behavioural proxy
    createdAt: number;
    lastAccessed: number;
}

// Tensor Impact Output
interface TensorImpact {
    axisDrift: Record<Axis, number>;
    psiLoad: number; //  $\Psi(t)$  normalised 0-1
    psiComponents: {
        emotionalDysreg: number;
        dissociation: number;
        insomnia: number;
        relationalAvoidance: number;
        hypervigilance: number;
    };
    omegaContribution: number;
    kappaModifier: number; // convergence surface sharpness adjustment
}

// Red Zone /  $\Phi(t)$  Evaluation
interface RedZoneStatus {
    phiThreshold: number; // 0.0-1.0
    exceeded: boolean;
    triggerAxis: Axis | null;
    vulnerabilityIndexRising: boolean;
    recoveryMomentumStalled: boolean;
    recommendedMode: CompanionMode;
}

```

```

// Semantic Context for LLM (No literals)
interface SemanticContext {
    harmModality?: string; // e.g., "interpersonal", "institutional",
"medical"
    affectiveValence?: string; // e.g., "shame_high", "grief_acute",
"rage_suppressed"
    somaticSignature?: string; // e.g., "thoracic_constriction",
"jaw_tension", "numb_flat"
    relationalPattern?: string; // e.g., "over_functioning",
"tethering_volatile", "avoidant"
    cognitiveDistortion?: string; // e.g., "personalisation",
"catastrophising", "fragmented"
    intensity: number; // 0.0-1.0
    regulatoryState: RegulatoryState;
    userRequest: CompanionMode;
    ndAdaptations: {
        alexithymiaProxy: boolean;
        externalSensoryAnchor: boolean;
        avoidInteroceptivePrompt: boolean;
    };
}

// Final Tokenised Payload (What crosses the boundary)
interface TokenisedPayload {
    userId: string; // derived, never stored with content
    timestamp: number;
    tokenisedText: string;
    semanticContext: SemanticContext;
    tensorImpact: TensorImpact;
    redZoneStatus: RedZoneStatus;
    companionDirectives: {
        sovereigntyMode: true;
        noUnsolicitedRegulation: true;
        cognitiveStyleRespect: true;
        presenceDefault: true;
    };
}

```

9.3 Primary API Interface

typescript

```
declare class PromptShieldTokeniser {
    /**
     * Initialises the tokeniser with secure enclave access and user key derivation.
     */
    static initialise(config: {
        masterKeyDerivationPath: string;
        enclaveType: 'ios_keychain' | 'android_keystore' | 'hardware_se';
        schemaVersion: '0.2';
    }): Promise<PromptShieldTokeniser>;
    /**
     * Core tokenisation method. Accepts raw input, returns structured payload
     *
     * Literal text is NEVER returned. Secure memory zeroed post-call.
     */
    async tokenise(input: string, config: {
        privacyTier: PrivacyTier;
        currentRegulatoryState?: RegulatoryState;
        requestedMode?: CompanionMode;
    }): Promise<TokenisedPayload>;
    /**
     * Updates the local mapping table. Used for user-assisted tagging or NER corrections.
     */
    async updateMapping(entry: {
        literal: string;
        token: string;
        contextHint?: string;
        axisWeights?: Partial<Record<Axis, number>>;
    }): Promise<void>;
    /**
     * Exports encrypted mapping blob for user-controlled backup.
     */
    async exportMapping(): Promise<{
        encryptedBlob: Uint8Array;
        checksum: string;
        exportTimestamp: number;
    }>;
    /**
```

```

    * Restores mapping from user-held encrypted backup.
    */
    async importMapping(encryptedBlob: Uint8Array, masterKey: string): Promise
<void>;
    /**
    * Clears volatile memory and resets session state.
    * Called on app background, logout, or explicit user action.
    */
    async clearSession(): Promise<void>;
    /**
    * Returns current mapping table size and integrity status.
    */
    async getMappingHealth(): Promise<{
        entryCount: number;
        integrityValid: boolean;
        lastSync: number;
        corruptionFlags: string[];
    }>;
}

```

9.4 Event & Hook System

typescript

```

type ShieldEvent =
    | 'tokenMapped'
    | 'redZoneTriggered'
    | 'ndFallbackActivated'
    | 'mappingCorrupted'
    | 'sessionCleared';

declare class PromptShieldTokeniser {
    on(event: ShieldEvent, handler: (data: any) => void): void;
    off(event: ShieldEvent, handler: (data: any) => void): void;
    // Event Payload Examples:
    // tokenMapped: { literal: string, token: string, tier: 'T1'|'T2'|'T3', axis: Axis }
    // redZoneTriggered: { phi: number, exceeded: true, triggerAxis: Axis, recommendedMode: string }
    // ndFallbackActivated: { reason: 'alexithymia' | 'sensory_overload', proxyUsed: string }
    // mappingCorrupted: { sector: string, recoveryAttempted: boolean, userActionRequired: boolean }
}

```

}

9.5 Atlas Framework Integration Mapping

Atlas Component	Tokeniser Integration Point
$\Psi(t)$ Symptom Load	psiLoad computed on-device via $w \times \text{Chronicity}^{1.2} \times \text{Volatility}^{0.8} \times \text{AxisWeight} \times 100$
Ω Trauma Residual	omegaContribution added to running Ω integral; ndProxyFlag adjusts decay eligibility
$\Phi(t)$ Present Impact	phiThreshold = $\Psi(t) \times I(t) \times (1 + \text{RD})$; red zone flags returned to companion without content
7-Axis Engine	axisDrift maps tokenised spans to REL, OCCF, EREG, CC, SOM, VIC, NI weights
C(t) Diary Card	Tokeniser consumes C(t) inputs, enriches semanticContext, updates tensor state
Companion Ten Commitments	companionDirectives enforce sovereigntyMode, noUnsolicitedRegulation, presenceDefault
ND Adaptations (INEHF)	ndAdaptations flags suppress interoceptive prompts, enable external sensory proxies, respect alexithymic tracking
Red Zone Architecture	redZoneStatus aligns with Tier 1–4 protocol; companion enters holding posture without content exposure

9.6 Usage Flow Example

typescript

```
const shield = await PromptShieldTokeniser.initialise({
  masterKeyDerivationPath: 'argon2id://user_passphrase',
  enclaveType: 'ios_keychain',
  schemaVersion: '0.2'
});

const payload = await shield.tokenise(
  "My stepfather abused me from age 6. I still get this tightness in my chest when I think about it, and I just feel so worthless.",
  {
    privacyTier: 'maximum',
    currentRegulatoryState: 'hyperaroused',
    requestedMode: 'witness'
  }
);

// What gets sent to LLM Intelligence Layer:
console.log(JSON.stringify({
  tokenised: payload.tokenisedText,
  context: payload.semanticContext,
  tensor: payload.tensorImpact,
  redZone: payload.redZoneStatus,
```

```

    directives: payload.companionDirectives
  }, null, 2));

```

9.6.1 LLM Receives:

```

json
{
  "tokenised": "Entity_A [ENT]_[CAREGIVER]_[T1]_[MALE] caused [HARM]_[INTERP
PERSONAL]_[T2]_[CHRONIC] to Subject during [TIME]_[AGE_RANGE]_[T1]_[EARLY_CHI
LD]. Somatic: [SOM]_[THORAX]_[T2]_[CONSTRICITION]. Affective: [AFF]_[SHAME]_[
T2]_[HIGH]. Cognitive: [COG]_[PERSONALISATION]_[T2].",
  "context": {
    "harmModality": "interpersonal",
    "somaticSignature": "thoracic_constriction",
    "affectiveValence": "shame_high",
    "cognitiveDistortion": "personalisation",
    "intensity": 0.92,
    "regulatoryState": "hyperaroused",
    "userRequest": "witness",
    "ndAdaptations": { "alexithymiaProxy": false, "avoidInteroceptivePrompt"
: true }
  },
  "tensor": {
    "psiLoad": 0.78,
    "omegaContribution": 0.14,
    "axisDrift": { "REL": -0.45, "VIC": -0.60, "EREG": -0.52, "SOM": -0.35 }
  },
  "redZone": {
    "phiThreshold": 0.61,
    "exceeded": false,
    "recommendedMode": "presence"
  },
  "directives": {
    "sovereigntyMode": true,
    "noUnsolicitedRegulation": true,
    "cognitiveStyleRespect": true,
    "presenceDefault": true
  }
}

```

9.7 Implementation Notes

Consideration	Recommendation
NER Engine	Start with rule-based regex + lightweight spaCy/DistilBERT mobile. Graduate to fine-tuned CPTSD/ND NER model in v0.3.
Memory Safety	Use mlock() on Unix, SecuredBuffer on Android/iOS. Zero memory after tokenise() returns.
Mapping Table Size	Index with B-Tree or hash map. Cap at 10k entries per user; archive oldest to encrypted cold storage if needed.
ND Fallback Logic	If user flags alexithymiaProxy: true, replace [SOM]_[INTERNAL] tokens with [SOM]_[EXTERNAL_PROXY] (e.g., HRV spike, sleep disruption, wearable telemetry).
Companion Prompt Injection	The directives object should be serialised into the system prompt exactly as: ## Operational Constraints\n- Sovereignty Mode: ACTIVE\n- No unsolicited regulation: ENFORCED\n- Default: Presence/Witness\n- ND Adaptations: {flags}

